Task 1. Short note (4–5 lines)
Kubernetes orchestration helps applications like Zomato and Swiggy manage thousands of food orders simultaneously. It automatically scales application pods according to user traffic. It distributes requests using Kubernetes Services and helps prevent server overload. If a container fails, Kubernetes can restart it automatically. This improves application availability, reliability, and performance.

Task 2. Create a multi-container Deployment

Create a file:

`spotify-playlist-deployment.yaml`

```
None
apiVersion: apps/v1
kind: Deployment

metadata:
  name: spotify-playlist

spec:
  replicas: 1

  selector:
    matchLabels:
      app: spotify-playlist

  template:
    metadata:
      labels:
        app: spotify-playlist
```

```
    spec:
      containers:

        - name: playlist-api
          image: nginx:latest
          ports:
            - containerPort: 80
```

Task 3. Apply and verify Deployment

Apply:

```
None
kubectl apply -f spotify-playlist-deployment.yaml
```

Check deployments:

```
None
kubectl get deployments
```

Check pods:

```
None
kubectl get pods
```

Describe the pod:

```
None
kubectl describe pod POD-NAME
```

You should see two containers:

```
None
playlist-api
song-database
```

Check container status:

```
None
kubectl get pods
```

Task 4. Create Kubernetes Service

Create:

`spotify-playlist-service.yaml`

```
None
apiVersion: v1
kind: Service

metadata:
  name: spotify-playlist-service

spec:
  type: ClusterIP

  selector:
    app: spotify-playlist

  ports:
```

```
    - name: playlist-api
      port: 8080
      targetPort: 80
      protocol: TCP
```

Apply:

```
None
kubectl apply -f spotify-playlist-service.yaml
```

Check:

```
None
kubectl get services
```